08. The service split: Core and Vendor-admin

Status	Draft v1.0. **A focused companion to `architecture.md`** (which holds the full design). This document records the split, why it was redrawn, what each service must never do, and a checklist for engineers building across the boundary
Date	2026-10-20
Depends on	`architecture.md`, `adr/0007-core-and-vendor-admin-services.md`, `adr/0008-commerce-in-core.md`

1. The decision in one paragraph

The team agreed that the vendor and admin side is a **separate FastAPI service**. The first draft of that split also put **orders and payments** in it, with Razorpay behind a seller back-office. That was wrong for an e-commerce platform: checkout and payment are part of the customer's buying path. In this version **Core (Django) owns commerce** (cart, checkout, orders, stock, payments, refunds, returns) and the **Vendor-admin service (FastAPI) is only the seller back-office** (vendors, catalog authoring, fulfilment screens, statements, admin tools). See ADR 0008.

What changed from the first draft	Before	Now
Name of the FastAPI service	"Marketplace"	**Vendor-admin service**
Orders, order state machine	Marketplace	**Core**
Payments, Razorpay, webhooks, refunds	Marketplace	**Core**
Stock counts	Marketplace (decrement on vendor confirm)	**Core** (reserved at order placement)
Checkout dependency	Core called the marketplace synchronously	**None**: checkout is one Core transaction
Who calls whom (commands)	Core to marketplace	**Vendor-admin to Core** (confirm, ship, cancel, deliver, collect, refund, set stock)
Database names	`shazlo_core`, `shazlo_market`	`shazlo_core`, **`shazlo_vendor`**
Hostnames	`market-api`	**`vendor-api`**, `admin`
Order copy	`order_refs` in core	**`vendor_orders`** in the vendor-admin database

2. The boundary

> Money, stock and order state are changed **only by Core**. The vendor-admin service never takes a payment, never changes a stock count and never owns an order.

Question	Answer
Where does a customer's order get created?	Core, in one database transaction with the stock reservation
Where is Razorpay called?	Core only; the payment credentials exist only in Core's settings
Who confirms, ships, cancels or delivers an order?	A vendor or admin through the vendor-admin service, which sends a signed command to Core; Core changes the order

Question	Answer
Who sets stock?	The vendor in the portal; the vendor-admin service sends the new `on_hand` to Core; Core owns the count
Who sets prices?	The vendor-admin service (price master); Core receives them by event and snapshots the catalog version on each order
Who sets commission?	The vendor-admin service; Core receives the rate by event and snapshots it on each order
Who computes statements?	The vendor-admin service, from its copy of orders
What happens if the vendor-admin service is down?	Customers browse, order and pay normally; vendors cannot use the portal; events wait in Core's outbox
What happens if Core is down?	The app is down; the portal shows its last copy of orders, and commands fail with a clear error

3. Communication in brief

Commands (vendor-admin to Core, synchronous, signed, idempotent) and events (both directions, outbox and inbox, at least once) are specified in `architecture.md` section 5 and `05-api-design.md` section 8. No shared database. Contracts live in the portal repo's `contracts/`, with a pinned copy in the core repo.

4. Builder's checklist

Before merging a change that touches both services, check:

1. Does any code in the vendor-admin service change a payment, a stock count or an order row? It must not; add a command to Core instead.
2. Does any code in Core read or write a vendor-admin table? It must not; use the catalog copy or an event.
3. Is the new command idempotent, and does it return the new state and version?
4. Is a new event add-only, versioned, and de-duplicated by `event_id`?
5. Does the contract change have a matching pull request in the other repository, reviewed by both owners?
6. Is the receiver deployed before the sender?
7. Does every Core command record the calling actor (`X-Actor`) in the audit log?

5. Impact on the MVP

Track	Owns	Notes
Track 1	Core hardening **and commerce**: checkout, stock reservation, order state machine, payments built behind the switch, internal commands, events	The largest track; checkout is one transaction; the cut order is in `06` and `architecture.md` section 14
Track 2	Vendor-admin service and portal: vendors, catalog authoring, order view, fulfilment screens, admin pages, SQLAdmin	Builds against the generated OpenAPI file and a stub Core
Track 3	App and feed	Order screens call Core only

Schedule, scope, definition of done and risks: `06-mvp-slice-and-migration.md`.

6. Decisions

ID	Decision	Choice
D1	Where commerce lives	**Core** (supersedes the earlier "marketplace")
D2	Database isolation	Two databases on one server
D3	Vendor-admin stack	FastAPI, SQLAlchemy 2, Alembic, SQLAdmin, ARQ later
D4	Event transport	Database outbox plus signed HTTP; broker later if needed
D5	Identity	Separate vendor and admin accounts in the vendor-admin service
D6	Portal	React SPA; server-rendered fallback only if behind schedule
D7	Hosting	One small VM, Docker Compose, Cloudflare in front
D8	Repositories	`shaz` (app and Core); portal repo (frontend, vendor-admin backend, contracts)
D9	Payments	Cash and UPI on delivery at launch; Razorpay in Core behind a switch
D10	Stock owner	Core
D11	Money flow for online payments	**Open (Q37)**: collect and pay out versus Razorpay Route